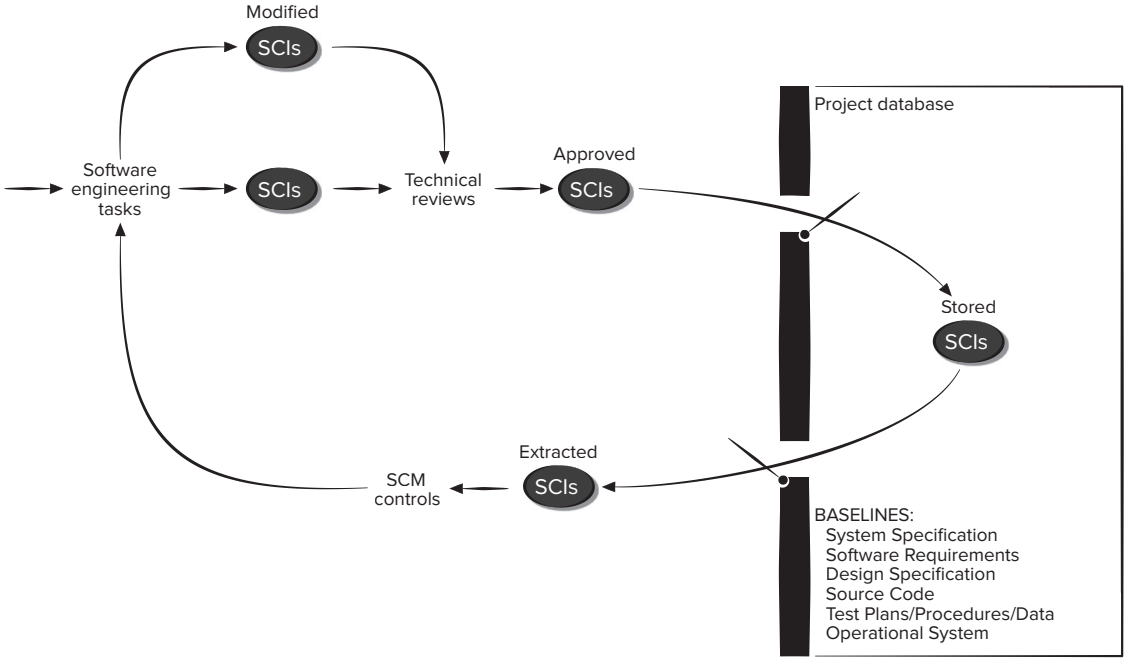
FIGURE 22.2 Baselined SCIs and the project database



name, attributes, and is "connected" to other objects by relationships. Referring to Figure 22.3, the configuration objects, **DesignSpecification, DataModel, ComponentN, SourceCode,** and **TestSpecification** are each defined separately. However, each of the objects is related to the others as shown by the arrows. A curved arrow indicates a compositional relation. That is, **DataModel** and **ComponentN** are part of the object **DesignSpecification.** A double-headed straight arrow indicates an interrelationship. If a change were made to the **SourceCode** object, the interrelationships enable you to determine what other objects (and SCIs) might be affected.[2]

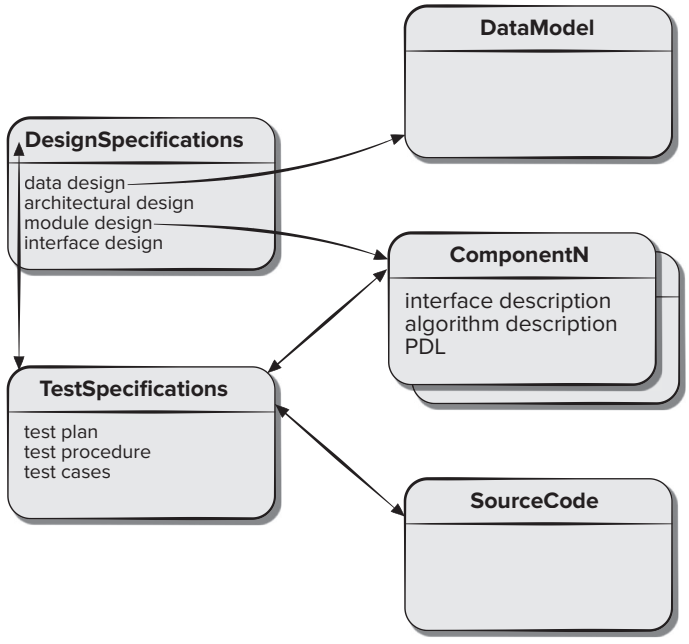
22.1.5 Management of Dependencies and Changes

We introduced the concept of traceability and the use of traceability matrices in Section 7.2.6. The traceability matrix is one way to document dependencies among requirements, architectural decisions (Section 10.5), and defect causes (Section 17.6). These dependencies need to be considered when determining the impact of a proposed change and guiding the selection test cases that should be used for regression testing (Section 20.3). de Sousa and Redmiles write that viewing dependency management as impact management[3] helps developers to focus on how changes made affect their work [Sou08].

2 These relationships are defined within the database. The structure of the database (repository) is discussed in greater detail in Section 22.2.

3 Impact management is discussed further in Section 22.5.2.

FIGURE 22.3
Configuration objects



Impact analysis focuses on organizational behavior as well as individual actions. Impact management involves two complementary aspects: (1) ensuring that software developers employ strategies to minimize the impact of their colleagues' actions on their own work, and (2) encouraging software developers to use practices that minimize the impact of their own work on that of their colleagues. It is important to note that when a developer tries to minimize the impact of her work on others, she is also reducing the work others need to do to minimize the impact of her work on theirs [Sou08].

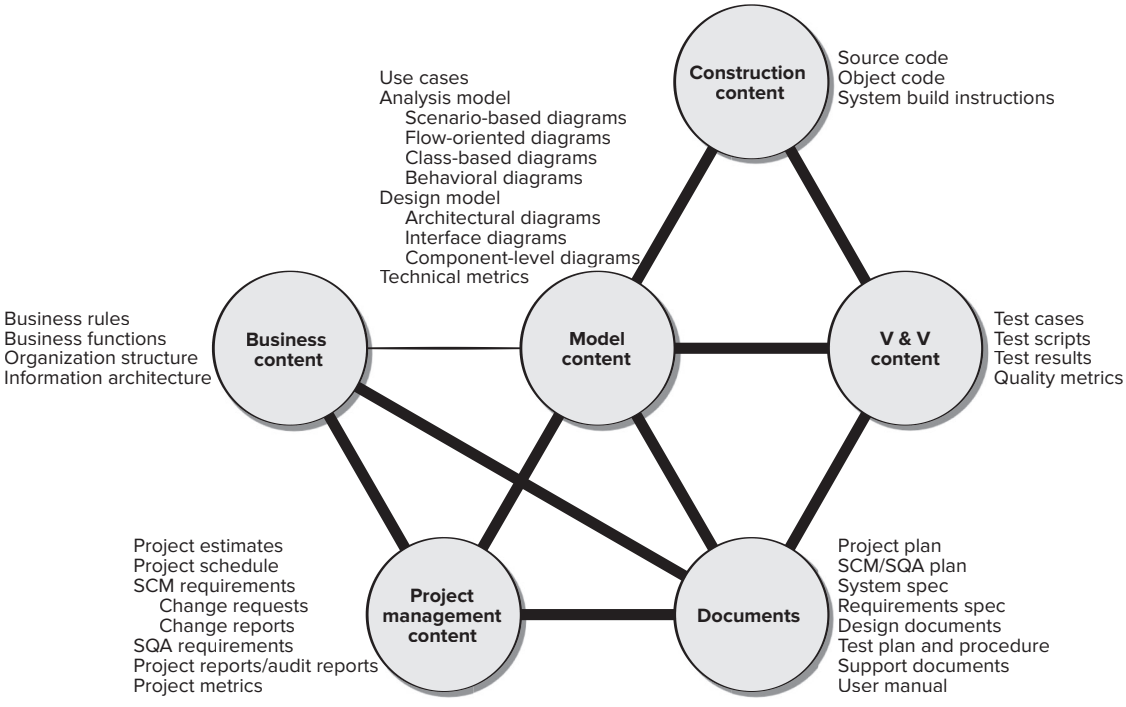
It is important to maintain software work products to ensure that developers are aware of the dependencies among the SCIs. Developers must establish discipline when checking items in and out of the SCM repository and when making approved changes, as discussed in Section 22.2.

22.2 THE SCM REPOSITORY

The SCM repository is the set of mechanisms and data structures that allow a software team to manage change in an effective manner. It provides the obvious functions of a modern database management system by ensuring data integrity, sharing, and integration. In addition, the SCM repository provides a hub for the integration of software tools, is central to the flow of the software process, and can enforce uniform structure and format for software engineering work products.

To achieve these capabilities, the repository is defined in terms of a meta-model. The *meta-model* determines how information is stored in the repository, how data

FIGURE 22.4 Content of the repository



can be accessed by tools and viewed by software engineers, how well data security and integrity can be maintained, and how easily the existing model can be extended to accommodate new needs.

22.2.1 General Features and Content

The features and content of the repository are best understood by looking at it from two perspectives: what is to be stored in the repository and what specific services are provided by the repository. A detailed breakdown of types of representations, documents, and other work products that are stored in the repository is presented in Figure 22.4.

A robust repository provides two different classes of services: (1) the same types of services that might be expected from any sophisticated database management system and (2) services that are specific to the software engineering environment.

A repository that serves a software engineering team should also (1) integrate with or directly support process management functions, (2) support specific rules that govern the SCM function and the data maintained within the repository, (3) provide an interface to other software engineering tools, and (4) accommodate storage of sophisticated data objects (e.g., text, graphics, video, audio).

22.2.2 SCM Features

To support SCM, the repository must be capable of maintaining SCIs related to many different versions of the software. More important, it must provide the mechanisms

for assembling these SCIs into a version-specific configuration. The repository tool set needs to provide support for the following features.

Versioning. As a project progresses, many versions (Section 22.5.2) of individual work products will be created. The repository must be able to save all these versions to enable effective management of product releases and to permit developers to go back to previous versions during testing and debugging.

The repository must be able to control a wide variety of object types, including text, graphics, bit maps, complex documents, and unique objects such as screen and report definitions, object files, test data, and results. A mature repository tracks versions of objects with arbitrary levels of granularity; for example, a single data definition or a cluster of modules can be tracked.

Dependency Tracking and Change Management. The repository manages a wide variety of relationships among the data elements stored in it. These include relationships between enterprise entities and processes, among the parts of an application design, between design components and the enterprise information architecture, between design elements and deliverables, and so on. Some of these relationships are merely associations, and some are dependencies or mandatory relationships.

The ability to keep track of all these relationships is crucial to the integrity of the information stored in the repository and to the generation of deliverables based on it, and it is one of the most important contributions of the repository concept to the improvement of the software development process. For example, if a UML class diagram is modified, the repository can detect whether related classes, interface descriptions, and code components also require modification and can bring affected SCIs to the developer's attention.

Requirements Tracing. This special function depends on link management and provides the ability to track all the design and construction components and deliverables that result from a specific requirements specification (forward tracing). In addition, it provides the ability to identify which requirement generated any given work product (backward tracing).

Configuration Management. A configuration management facility keeps track of a series of configurations representing specific project milestones or production releases.

Audit Trails. An audit trail establishes additional information about when, why, and by whom changes are made. Information about the source of changes can be entered as attributes of specific objects in the repository. A repository trigger mechanism is helpful for prompting the developer or the tool that is being used to initiate entry of audit information (such as the reason for a change) whenever a design element is modified.

22.3 VERSION CONTROL SYSTEMS

Version control combines procedures and tools to manage different versions of configuration objects that are created during the software process. A version control system implements or is directly integrated with four major capabilities: (1) a project

database (*repository*) that stores all relevant configuration objects, (2) a *version management* capability that stores all versions of a configuration object (or enables any version to be constructed using differences from past versions), (3) a *make facility* that enables you to collect all relevant configuration objects and construct a specific version of the software. In addition, version control and change control systems often implement (4) an *issues tracking* (also called *bug tracking*) capability that enables the team to record and track the status of all outstanding issues associated with each configuration object.

A number of version control systems establish a *change set*—a collection of all changes (to some baseline configuration) that are required to create a specific version of the software. Dart [Dar91] notes that a change set "captures all changes to all files in the configuration along with the reason for changes and details of who made the changes and when."

A number of named change sets can be identified for an application or system. This enables you to construct a version of the software by specifying the change sets (by name) that must be applied to the baseline configuration. To accomplish this, a *system modeling* approach is applied. The system model contains: (1) a *template* that includes a component hierarchy and a "build order" for the components that describes how the system must be constructed, (2) construction rules, and (3) verification rules.[4]

A number of different automated approaches to version control have been proposed over the years.[5] The primary difference in approaches is the sophistication of the attributes that are used to construct specific versions and variants of a system and the mechanics of the process for construction.

22.4 CONTINUOUS INTEGRATION

Best practices for SCM include: (1) keeping the number of code variants small, (2) test early and often, (3) integrate early and often, and (4) tool use to automate testing, building, and code integration. *Continuous integration* (CI) is important to agile developers following the DevOps workflow (Section 3.5.3). CI also adds value to SCM by ensuring that each change is promptly integrated into the project source code, compiled, and tested automatically. CI offers development teams several concrete advantages [Mol12]:

Accelerated feedback. Notifying developers immediately when integration fails allows fixes to be made while the number of performed changes is small.

Increased quality. Building and integrating software whenever necessary provides confidence into the quality of the developed product.

4 It is also possible to query the system model to assess how a change in one component will impact other components.

5 Github (https://github.com/), Perforce (https://www.perforce.com/), and Apache Subversion also known as SVN (http://subversion.apache.org/) are popular version control systems.

Reduced risk. Integrating components early avoids risking a long integration phase because design failures are discovered and fixed early.

Improved reporting. Providing additional information (e.g., code analysis metrics) allows for more accurate configuration status accounting.

CI is becoming a key technology as software organizations begin their shift to more agile software development processes. CI is best done using specialized tools.[6] CI allows project managers, quality assurance managers, and software engineers to improve software quality by reducing the likelihood of defects escaping outside the development team. Early defect capture always reduces the development costs by allowing cheaper fixes earlier in the software project time line.

22.5 THE CHANGE MANAGEMENT PROCESS

The software change management process defines a series of tasks that have four primary objectives: (1) to identify all items that collectively define the software configuration, (2) to manage changes to one or more of these items, (3) to facilitate the construction of different versions of an application, and (4) to ensure that software quality is maintained as the configuration evolves over time.

A process that achieves these objectives need not be bureaucratic and ponderous, but it must be characterized in a manner that enables a software team to develop answers to a set of complex questions:

- How does a software team identify the discrete elements of a software configuration?
- How does an organization manage the many existing versions of a program (and its documentation) in a manner that will enable change to be accommodated efficiently?
- How does an organization control changes before and after software is released to a customer?
- How does an organization assess the impact of change and manage the impact effectively?
- Who has responsibility for approving and ranking requested changes?
- How can we ensure that changes have been made properly?
- What mechanism is used to apprise others of changes that are made?

These questions lead to the definition of five SCM tasks—identification, version control, change control, configuration auditing, and reporting—illustrated in Figure 22.5.

6 Puppet (https://puppet.com/), Jenkins (https://jenkins.io/), and Hudson (http://hudson-ci.org/) are examples of CI tools. Travis-CI (https://travis-ci.org/) is a CI tool designed for sync projects residing on Github.